

RESEARCH ARTICLE | FEBRUARY 05 2025

A systematic dataset generation technique applied to data-driven automotive aerodynamics

Mark Benjamin  ; Gianluca Iaccarino 

APL Mach. Learn. 3, 016110 (2025)

<https://doi.org/10.1063/5.0233367>

Articles You May Be Interested In

DrivAer transformer: A high-precision and fast prediction method for vehicle aerodynamic drag coefficient based on the DrivAerNet++ dataset

Physics of Fluids (July 2025)

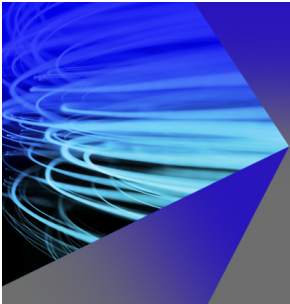
TripOptimizer: Generative three-dimensional shape optimization and drag prediction using triplane variational autoencoder networks

Physics of Fluids (December 2025)

Numerical investigation of wake flow effects on particulate matter dispersion characteristics based on the DrivAer Estateback model

Physics of Fluids (September 2025)

23 July 2026 21:44:29



AIP Advances

Why Publish With Us?

**21DAYS**
average time
to 1st decision

**OVER 4 MILLION**
views in the last year

**INCLUSIVE**
scope

[Learn More](#)



A systematic dataset generation technique applied to data-driven automotive aerodynamics

Cite as: APL Mach. Learn. 3, 016110 (2025); doi: 10.1063/5.0233367

Submitted: 14 August 2024 • Accepted: 20 January 2025 •

Published Online: 5 February 2025



Mark Benjamin^{a)}  and Gianluca Iaccarino^{b)} 

AFFILIATIONS

Center for Turbulence Research, 481 Panama Mall, Stanford, California 94305, USA

^{a)} Author to whom correspondence should be addressed: markben@stanford.edu

^{b)} jiops@stanford.edu

ABSTRACT

A novel strategy for generating datasets has been developed within the context of drag prediction for automotive geometries using neural networks. A primary challenge in this field is constructing a training database of sufficient size and diversity. Our method relies on a small number of initial data points and provides a recipe to systematically interpolate between them, generating an arbitrary number of samples at the desired quality. We tested this strategy using a representative automotive geometry and demonstrated that convolutional neural networks perform exceptionally well at predicting drag coefficients and surface pressures. Promising results were obtained in testing extrapolation performance. Our method can be applied to other problems of aerodynamic shape optimization.

© 2025 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC BY-NC-ND) license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>). <https://doi.org/10.1063/5.0233367>

I. INTRODUCTION

A. Motivation

The computation of aerodynamic drag is an expensive component of automotive design, requiring one computational fluid dynamics simulation per design point. It is also an essential component of the design process; since the drag force is quadratic in vehicle velocity, it becomes the dominant mode of energy loss at high speed, affecting vehicle top speed, fuel efficiency, and stability. The complexity of modern vehicle designs leads to a high-dimensional parameter space that can be prohibitively expensive to sample. In addition, the evaluation of the quantity of interest—the drag coefficient—for each single design point is costly, requiring the solution of the Navier–Stokes equations at high Reynolds numbers, on a mesh sufficiently fine to resolve the details of the automotive design. Surrogate models are therefore of interest; in particular, given the nature of the governing equations and the complexity of the geometries of interest, one seeks data-driven models for drag prediction. The drag coefficient depends on flow and geometry parameters; in our study, the flow parameters are fixed; thus, the model learns a mapping between the automotive geometry and the drag coefficient.

The primary issue with constructing data-driven models in this application space is the paucity of freely available automotive

geometries, owing primarily to the proprietary nature of commercial designs. In addition, automotive design involves not just aerodynamic efficiency but other, and sometimes conflicting, considerations such as aesthetics, regulatory compliance, and manufacturability. This is in contrast to, for example, airplane wing design, where (regulatory compliance and manufacturability notwithstanding) the design process is far more organic: one begins with a cross-section of an airfoil with the desired drag and lift characteristics, then the wing is extruded in the third dimension based on loading characteristics. Thus, in practice, the design problem for the aerodynamicist involves a limited scope; small variations in a discrete number of parameters. For example, the aerodynamicist will never be able to redesign the Ford Mustang to have the same drag coefficient as, say, a Tesla Model S (which is one of the only passenger cars whose exterior design was driven primarily by aerodynamics¹), because a Ford Mustang's overall shape and first-order drag characteristics are dictated by the need to look like a Ford Mustang, for aesthetic appeal. This design paradigm makes it all the more important to have a rich and variegated training set, because it is the details on a scale much smaller than the characteristic length scale of the vehicle that one has the freedom to modify with the aim of improving aerodynamics. In the absence of a sufficiently large and diverse training set, models suffer in accuracy and

predictive capability. In this work, we develop a methodology of generating geometries for training in a controlled manner, starting with a small number of starting cases. By generating a dataset with sufficient granularity, we show that data-driven model predictions show excellent predictive capabilities for quantities of interest at two different levels of abstraction—scalar (the drag coefficient) and tensor (the surface pressure coefficient on the wetted area).

B. Geometry parameterization and dataset generation

There have been attempts in the literature to address the issue of database generation in aerodynamics problems. In this work, the convention is adopted that the term “database” refers to representations of the geometries, whereas the term “dataset” refers to pairings of geometries with corresponding aerodynamics quantities of interest supplied by simulations or experiments, used for training data-driven surrogates. Viquerat and Hachem² studied the problem of 2D airfoils in the laminar ($Re\ 10$) regime and used a random process to generate Bézier curves with which airfoils were developed. Their approach—which used an immersed boundary solver—showed the importance of having sufficient variety in the training set but is not directly translatable to the 3D, fully turbulent automotive design problem for reasons described earlier, as one does not typically encounter realistic designs through a randomized morphing process. Garcia-Fernandez *et al.*³ studied simplified representations of heavy vehicles, using CNNs to predict aerodynamic quantities of interest. They followed a similar approach to the other works above, with a manual parameterization of the Ahmed body to generate a training set. Other parameterizations explored in the literature include using a PolyCube map to represent the geometry⁴ or using parameterized curves to generate two-dimensional silhouettes of cars,⁵ both of which require manual parameterization and have been tested on relatively simple shapes. Song *et al.*⁶ used the versatile ShapeNet database, which is a large repository of freely available Computer-Aided Design (CAD) models organized hierarchically, in their investigations of automotive drag.

The ShapeNet dataset⁷ deserves some attention here. It is a large freely available online repository of 3D models, labeled and organized by conceptual hierarchy; for instance, models labeled “car” are found under “vehicle.” The purpose of the repository is to serve as a large corpus of data to support research in computer graphics, computer vision, robotics, and other related disciplines. The source for most of these models is various openly accessible CAD repositories online; thus, the models are of limited quality, some more reminiscent of early benchmark cases than modern realistic designs. However, what the repository lacks in quality it makes up for in quantity: there are about 8000 car models of varied description, of which a selection is shown in Fig. 1. The ShapeNet dataset has been used in the vast majority of studies to provide training data for machine learning-based aerodynamics predictors.^{4,6–11} The major limitation of ShapeNet is the lack of surface resolution in all of the models, so the resulting model cannot be considered a tool to aid even the initial stages of automotive aerodynamic design.

An approach taken in the literature has been to develop a database with manual parameterizations of the much higher quality DrivAer model discussed above.^{12,13} For example, Elrefaie *et al.*¹³ generate a database using 29 parameters identified across the entire DrivAer model. The drawbacks to this approach are twofold: first, manual parameterization is labor- and time-intensive; second: there is no guarantee that the resulting database is an exhaustive representation of relevant shapes. This approach is thus not scalable beyond limited scope, such as, for example, predicting drag in minor variations of a specific model of automobile.

Virtually all the existing work in the literature uses RANS (Reynolds-Averaged Navier–Stokes) simulation data as the ground truth to train the data-driven models. While RANS simulations have been the workhorse for engineering computational fluid dynamics (CFD), they have two primary limitations: the extreme mesh requirements for grid-independent solutions in complex geometries, and the inability to capture inherently unsteady phenomena,¹⁴ such as the strong separation seen in the wake of a flow past a bluff automotive body. For these reasons, we use wall-modeled

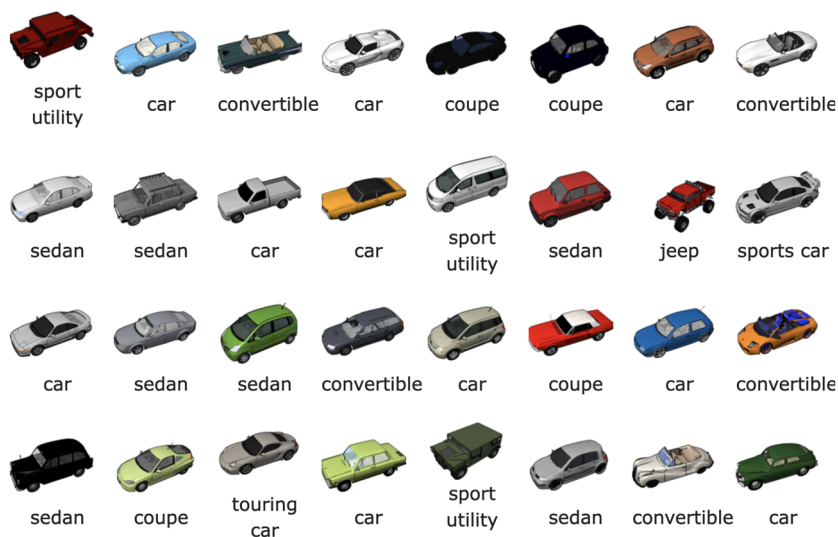


FIG. 1. A selection of ShapeNet automobile models.

large-eddy simulations (WMLES) to generate the flow solutions. Recent advances in this modeling approach¹⁵ have made the cost of accurate solutions of first-order mean quantities for complex cases of engineering interest tractable.^{16–18}

II. METHODOLOGY

In this section, we describe the dataset generation method we propose. The goal is to go from a handful of starting geometries and use them to generate several more in a controlled manner. This is done by constructing the convex hull of the signed distance functions (SDFs) of their surfaces. We use the simplest form of interpolation: barycentric, to generate the convex hull. Finally, we require that the process be bi-directional; that is, the format of the projections (intermediate geometries) is the same as that of the base cases. Each step in this process is described in detail below.

A. Geometry representation

We begin with a discretized representation of the geometry in the stereolithographic (STL) format. The unstructured nature of this format makes the generation of new geometries by interpolation between base cases challenging due to the absence of consistent mapping of corresponding points or features. However, an STL representation is more readily compatible with unstructured CFD

solvers, so we introduce an intermediate transformation to a structured representation and return to the STL format after any interpolation is performed. An STL represents the surface as a discrete triangulation, which makes mesh generation for CFD a lot simpler than dealing directly with the parametric Non-Uniform Rational B-Splines (NURBS), which are more commonly used in Computer-Aided Design (CAD). While the resolution of the STL representation can be controlled to better represent complex shapes, it does not provide the exact definition (down to computer precision) that NURBS achieve (see Fig. 2).

While the STL representation allows for straightforward generation of a fluid domain mesh, the format is unstructured, that is, it is merely a list of triangle vertices with associated normals, with no notion of structured connectivity between triangles. Although this is not a concern in unstructured mesh generation, it presents difficulties in the primary objective of this work, which is to generate new geometries by shape interpolation between given geometries. Given arbitrary STL discretizations of two shapes, there is no vertex correspondence between the two that can be established, which could serve for a basis for interpolation. For these reasons, it is convenient to use the Signed Distance Function (SDF),¹⁹ which is defined as the normal distance of each point in the grid to the surface of the geometry. Thus, the SDF is a field that has a value of zero on the surface, and is by convention positive for points outside the geometry and negative for points inside it. Since the distance field is smooth, it allows for effective interpolation; that is, given two SDFs $\phi_1(x_i)$ and $\phi_2(x_i)$, where x_i are spatial coordinates on a discrete, structured grid, a new SDF can be constructed as follows:

$$\phi_3(x_i) = \alpha\phi_1(x_i) + (1 - \alpha)\phi_2(x_i), \quad (1)$$

where α is a scalar. Applying Eq. (1) to a unit sphere and unit cube, both appropriately centered, one can produce a family of intermediate shapes by varying α , an example of which is shown in Fig. 3. While SDF interpolation does not produce many guarantees about the topology of the resulting shapes, it is straightforward to do for basis geometries (here, the starting cube and the sphere) of arbitrary complexity. This idea is leveraged in the dataset generation procedure proposed in this work.

The SDF representation of geometries is not commonly used in either CAD or CFD settings; thus, a method is developed below

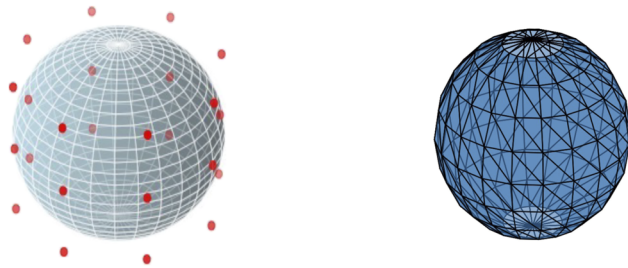


FIG. 2. Parametric and faceted representations of a sphere. Left: NURBS, showing control points (dots) from which the final shape is constructed. Right: STL, showing a relatively coarse triangulation.

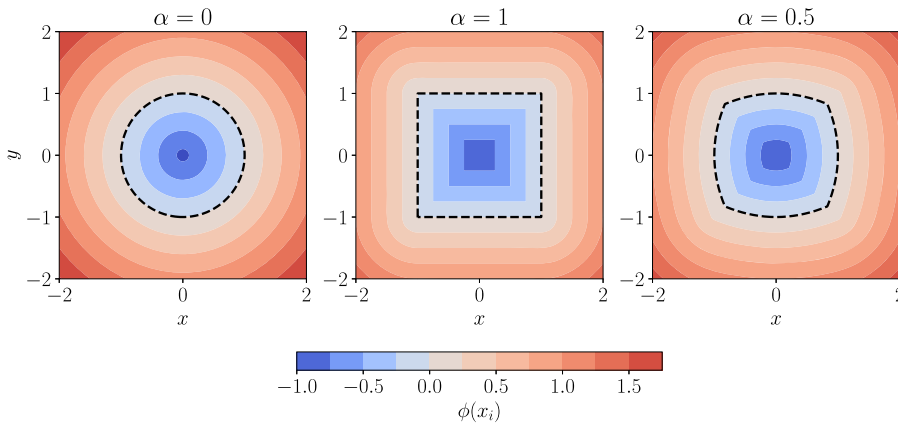


FIG. 3. SDF interpolation between a sphere and a cube, per Eq. (1). An x - y slice through the mid-plane is shown. Left: a pure sphere; center: a pure cube; right: an intermediate shape obtained by setting $\alpha = 0.5$.

that uses ray-tracing to convert an STL into its equivalent SDF. The dataset generation method proposed in this work begins with a number of starting (or basis) geometries, from which intermediate geometries are developed by shape interpolation of SDFs. The steps involved in this process are enumerated below.

- **Structured mesh initialization:** The bounding box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ is determined for the STL tessellation $\mathcal{T}$ by evaluating the axis-aligned bounds of all triangles $\{\Delta_i = (\mathbf{v}_1^i, \mathbf{v}_2^i, \mathbf{v}_3^i) \mid i \in \{1, \dots, M\}\}$, where M is the total number of triangles in the STL tessellation, and $\mathbf{v}_1^i, \mathbf{v}_2^i, \mathbf{v}_3^i \in \mathbb{R}^3$ are the vertices of the i -th triangle. The bounding box is discretized into a structured Cartesian grid $\mathbb{G}$ with dimensions $N_x \times N_y \times N_z$, where each N_i is the desired discretization resolution along each axis. It is straightforward to apply a localized refinement in discretization in regions where finer features need to be captured, but for simplicity, we use a uniform discretization resolution in the whole domain in this study.
- **Ray-tracing:** A set of rays $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is generated along each principal axis (x, y, z), where $\mathbf{o}$ is the ray origin and $\mathbf{d}$ is the unit direction vector aligned with the axis. The ray origins are placed at the centers of the bounding box faces, thus the total number of rays cast is $N_x \times N_y \times N_z$. For each ray $\mathbf{r}(t)$, the algorithm evaluates possible intersections with each triangle $\Delta_i \in \mathcal{T}$. The intersection test utilizes the Möller–Trumbore algorithm²⁰ to determine if and where the ray intersects the triangle, producing an intersection point $\mathbf{p}$ when a hit is detected. Figure 4 shows a schematic of the ray casting process for a sphere.

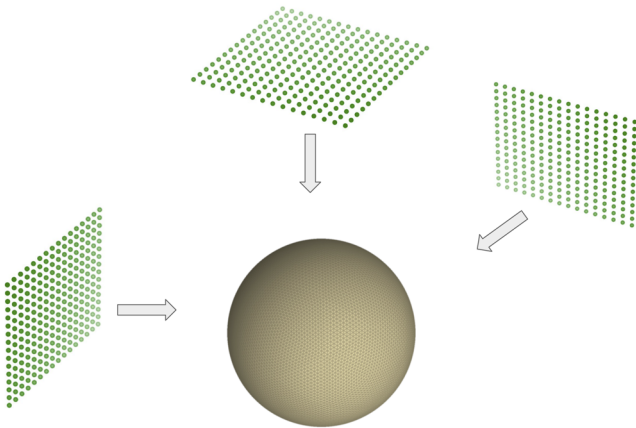


FIG. 4. Schematic of the ray-tracing for the STL of a sphere, showing three orthogonal sheets of rays. Each green dot represents the origin, $\mathbf{o}$, of a ray.

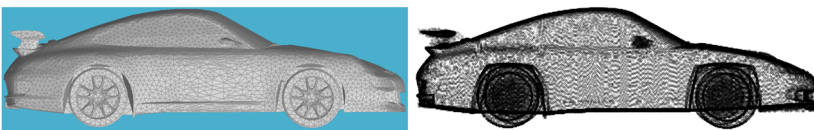


FIG. 5. Left: original STL; right: binary representation.

- **Intersect attribution:** Once an intersection point $\mathbf{p} = (p_x, p_y, p_z)$ is detected, it is mapped onto the corresponding grid cell in $\mathbb{G}$. The normalized coordinates,

$$\tilde{\mathbf{p}} = \left(\frac{p_x - x_{\min}}{x_{\max} - x_{\min}}, \frac{p_y - y_{\min}}{y_{\max} - y_{\min}}, \frac{p_z - z_{\min}}{z_{\max} - z_{\min}} \right),$$

are scaled to obtain the voxel indices (i, j, k) :

$$(i, j, k) = \lfloor \tilde{\mathbf{p}} \cdot (N_x - 1, N_y - 1, N_z - 1) \rfloor,$$

where the floor function $\lfloor \cdot \rfloor$ converts the continuous coordinates to discrete indices. The intersection grids from ray-tracing along all three principal axes are combined to yield a final binary grid $\mathbb{B} \in \{0, 1\}^{N_x \times N_y \times N_z}$, where each cell is defined by

$$\mathbb{B}_{i,j,k} = \begin{cases} 1 & \text{if } \exists \mathbf{r}(t) \cap \Delta_i \text{ s.t. } (i, j, k) = \left\lfloor \frac{\mathbf{p} - \mathbf{x}_{\min}}{\Delta_x} \right\rfloor, \\ 0 & \text{otherwise.} \end{cases}$$

Here, $\mathbb{B}_{i,j,k}$ represents whether voxel (i, j, k) in the grid contains part of the geometry (1) or is empty (0). This transformation yields a discrete voxelized representation of the continuous STL geometry $\mathcal{T}$ within the structured grid $\mathbb{G}$.

Figure 5 shows an example of the binary representation of an example geometry of a car, at a ray-tracing resolution of 1024^3 . Every fifth intersection is shown.

B. Geometry interpolation

With the binary representations constructed as above, we next look at the question of interpolating between n given geometries, $\mathbb{B}_{i,j,k}^1, \mathbb{B}_{i,j,k}^2, \dots, \mathbb{B}_{i,j,k}^n$. To accomplish this, we first convert the binary grids into Signed Distance Functions (SDFs), enabling a continuous representation of the geometries. For all binary grids $\mathbb{B}_{i,j,k}^i$, a hole-filling operation is applied to remove internal voids that are not connected to the boundary of the grid. This involves first dilating the complement of the binary grid $\bar{\mathbb{B}}_{i,j,k}^i = 1 - \mathbb{B}_{i,j,k}^i$:

$$\mathcal{D}^n[\bar{\mathbb{B}}_{i,j,k}^i],$$

where $\mathcal{D}^n$ represents the dilation operation iterated n times. The hole-filled grid is then the complement of this dilated result:

$$\bar{\mathbb{B}}_{i,j,k}^i = 1 - \mathcal{I}[\mathcal{D}^n[\bar{\mathbb{B}}_{i,j,k}^i]],$$

where $\mathcal{I}$ represents the invasion step, which identifies the regions that cannot be reached from the image boundary. For the filled

binary grid $\tilde{\mathbb{B}}_{i,j,k}^i$, the Signed Distance Function (SDF) $\phi_{i,j,k}^i$ is computed as the difference between the distance transform of the exterior and interior regions:

$$\phi_{i,j,k}^i = d(1 - \tilde{\mathbb{B}}_{i,j,k}^i) - d(\tilde{\mathbb{B}}_{i,j,k}^i),$$

where $d(\cdot)$ represents the Euclidean distance transform. The first term calculates the distance to the nearest external boundary, and the second term calculates the distance to the nearest internal boundary. The SDF $\phi_{i,j,k}^i$ thus yields positive values inside the geometry, negative values outside, and zero at the surface. The interpolation involves constructing the convex hull of the basis cases using a barycentric map over an n -simplex. Given the SDFs $\phi_{i,j,k}^i$ for each geometry $i \in \{1, \dots, n\}$, the interpolated SDF $\bar{\phi}_{i,j,k}(d)$ is computed as a weighted sum of the individual SDFs:

$$\bar{\phi}_{i,j,k}(d) = \sum_{i=1}^n w_i \phi_{i,j,k}^i(d),$$

subject to the constraints,

$$\sum_{i=1}^n w_i = 1,$$

and

$$w_i \geq 0 \quad \forall i \in \{1, \dots, n\}.$$

Here, the weights w_i are the barycentric coordinates corresponding to the location d within the n -simplex defined by the n input geometries. The resulting interpolated SDF $\bar{\phi}_{i,j,k}(d)$ provides a continuous scalar field that represents an interpolation of the original geometries within the simplex. Though the actual parameter space is high-dimensional, involving the morphing of a complex, three-dimensional surface, we are able to effectively parameterize this using the dimension of the simplex n .

C. Geometry reconstruction

After computing the interpolated SDF $\bar{\phi}_{i,j,k}(d)$ using the barycentric weights, the interface of the interpolated geometry is extracted. The interface corresponds to the numerical zero-level set of $\bar{\phi}_{i,j,k}(d)$, identified by

$$\Gamma_{i,j,k} = \begin{cases} 1 & \text{if } -\epsilon \leq \bar{\phi}_{i,j,k}(d) \leq \epsilon, \\ 0 & \text{otherwise,} \end{cases}$$

where ϵ is a small positive threshold that depends on the scale of the models. In our testing, we found that while this value had to be hand-tuned, it was relatively insensitive across the dataset.

Finally, we reconstruct the surface geometry of the interpolated object. This is achieved by applying the marching cubes algorithm²¹ to the binary scalar field derived from $\Gamma_{i,j,k}$. The marching cubes algorithm generates a triangulated mesh that approximates the surface $\partial\Omega$, where Ω is the region inside the geometry corresponding to $\Gamma_{i,j,k} = 1$. The vertices $\mathbf{v}_m$ and faces $\mathbf{f}_n$ of the resulting mesh $\mathcal{M}$ are constructed by interpolating the positions within the voxels where the interface $\Gamma_{i,j,k}$ intersects the grid. The mesh $\mathcal{M}$, representing the reconstructed geometry, can be expressed as

$$\mathcal{M} = \{\mathbf{v}_m, \mathbf{f}_n\} \quad \text{with} \quad \mathbf{v}_m \in \partial\Omega, \quad \mathbf{f}_n \subset \partial\Omega.$$

Here, $\mathbf{v}_m$ are the vertices positioned on the surface $\partial\Omega$ of the interpolated geometry and $\mathbf{f}_n$ are the faces connecting these vertices to form a closed surface. To refine the mesh and remove artifacts like sharp corners resulting from the marching cubes algorithm, an iterative Laplacian smoothing operation²² is applied, where each vertex $\mathbf{v}_m$ is adjusted based on the average position of its neighboring vertices $\mathbf{v}_m^{(i)}$:

$$\mathbf{v}_m' = \mathbf{v}_m + \lambda \sum_i (\mathbf{v}_m^{(i)} - \mathbf{v}_m),$$

where $\mathbf{v}_m'$ is the new position of the vertex after smoothing, and λ is a smoothing factor. It is evident that with an increased ray-tracing resolution, the reconstruction more closely approximates the fidelity of $\mathcal{T}$.

We have developed a software package that uses the above methodology to generate arbitrary-sized datasets. It is available to use on GitHub: <https://github.com/benjamark/ShapeCH>. The ray-tracing has been graphics processing unit (GPU)-accelerated and tested on up to 10×10^9 rays, which provides reconstruction at a sufficient resolution for a typical automotive model.

D. Properties of the interpolated SDFs

The use of the SDFs $\phi_{i,j,k}(d)$ for interpolation provides advantages for generating shapes from which a robust reconstruction can be made. The properties that make this possible are enumerated below:

- **Feature preservation:** $\bar{\phi}_{i,j,k}(d)$ will not introduce new geometric or topological features that were not present in the basis SDFs $\phi_{i,j,k}^i(d)$. Since the interpolation is a convex operation, no new critical points or disconnected components are generated. For example, interpolating between two similar car SDFs will not introduce a hole or an additional disconnected surface. The zero-level set $\{x : \bar{\phi}_{i,j,k}(x) = 0\}$ will reflect the features of the original zero-level sets.
- **Inclusion:** If one basis SDF $\phi_{i,j,k}^A(d)$ is fully contained within another $\phi_{i,j,k}^B(d)$, then the interpolated SDF $\bar{\phi}_{i,j,k}(d)$ will also be contained within $\phi_{i,j,k}^B(d)$ for all $k \in [0, 1]$. This inclusion property ensures that if one shape is nested inside another in the original SDFs, the interpolated shape remains within the boundary of the outer shape. For example, if $\phi_{i,j,k}^A(d)$ is the SDF of a sedan, and $\phi_{i,j,k}^B(d)$ is its estate equivalent, such that the rear roofline of the sedan is fully contained within that of the estate, the interpolation will produce shapes contained within the surface of the estate.
- **Continuity:** The interpolated SDF $\bar{\phi}_{i,j,k}(d)$ is a convex combination of the basis SDFs $\phi_{i,j,k}^i(d)$. Given that each $\phi_{i,j,k}^i(d)$ is discretely continuous, $\bar{\phi}_{i,j,k}(d)$ is also continuous. The continuity argument guarantees that for any level (c), the level set $\{x : \bar{\phi}_{i,j,k}(x) = c\}$ will vary continuously as (k) changes. The zero level set is not guaranteed to exist, but it is *likely* to exist when the basis SDFs have sufficient intersection, and are not completely disjoint, which can be ensured when positioning the basis STLs.

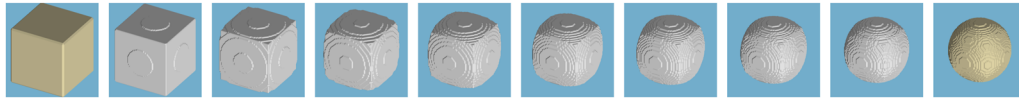


FIG. 6. Cube-to-sphere morph test. The ray-tracing resolution is 64 rays per dimension. The gold shapes are the basis (the cube and the sphere) and the silver shapes are the interpolated geometries.

E. Verification tests

Two verification tests are shown below that serve to both confirm the expected performance of the proposed method and illustrate a few interpolated shapes. The first test is the cube-to-sphere morph, a standard test in shape morphing and computer graphics. For this illustration, a ray-tracing resolution of 64 rays per dimension is chosen. The resulting shapes within the one-dimensional convex hull are shown in Fig. 6. The finite resolution of the ray-tracing introduces an additional length scale to the problem, which will need to be adjusted to resolve features of interest in the geometries.

For a more realistic test, the reconstruction of the DrivAer sedan model is shown. This test has only one basis case; thus, there is only one element in the convex hull, but it allows for visualization of the reconstruction quality of the method. A resolution of 1024^3 rays is used. The original and the reconstructed images are shown in Fig. 7. It is seen that the reconstruction faithfully captures the surface features of the original. There is a faint, spurious, low-amplitude rippling in certain regions of low curvature and detail, which is an artifact of the finite resolution of the ray-tracing. These attenuate with increasing resolution, just as errors in the STL representation of the underlying CAD curves attenuate with a finer triangulation. In this study, we simply ensure that these are small enough to not have a significant impact on the flow solution from the CFD.

III. SIMULATION AND DATA-DRIVEN MODEL DESIGN

A. Geometry and dataset

To best represent the complexities of a typical automobile, we consider the DrivAer model,²³ an open-source midsize passenger car geometry (Fig. 8), developed to assess the quality of and contrast automotive aerodynamics investigations carried out using computational fluid dynamics tools.²⁴ The model is available in three configurations: a fastback, estate, and sedan, shown in Fig. 9. Elrefaie *et al.*¹²



FIG. 8. Perspective view of the DrivAer geometry in notchback configuration.

used this geometry as a starting point, but only considered discrete, parametric modifications, and predicted the scalar drag coefficient using a point cloud representation.

While their dataset provides a high-quality set of realistic geometries, we construct a dataset organically by interpolating between the three DrivAer configurations to generate a family of designs, our only parameters being the choice of basis designs to interpolate between. The three configurations are different enough to provide a sufficient variation in drag coefficient (the separation aft of the vehicle being significantly impacted by the form of the rear roof slope), but also similar enough to provide a controlled testing ground for our proposed methodology.

IV. LARGE-EDDY SIMULATIONS

Large-eddy simulations (LES) are used as the numerical model to obtain the ground truth aerodynamic quantities of interest for the training dataset. Large-eddy simulations directly resolve the largest energy-containing scales of turbulent flows and model the small scales. This is achieved by applying a low-pass filter to the governing Navier–Stokes equations of motion, whereby scales smaller than the filter are removed. In common practice, high-frequency modes are removed implicitly by a combination of the finite sizing of the mesh and the filtering effect of numerical discretizations of

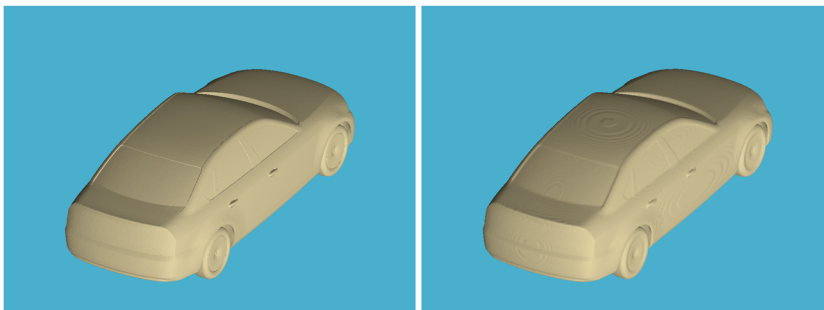


FIG. 7. Verification test of the DrivAer sedan model; left: original STL; right: reconstructed STL.

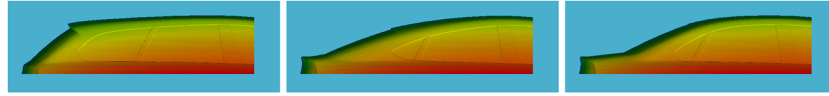


FIG. 9. DrivAer back configurations: estateback (left), fastback (center), and notchback (right).

the equations. The LES governing equations for an incompressible, single-phase fluid with constant fluid properties and no body forces are the continuity equation,

$$\frac{\partial \tilde{u}_i}{\partial x_i} = 0, \quad (2)$$

and the momentum equation,

$$\frac{\partial \tilde{u}_i}{\partial t} + \tilde{u}_j \frac{\partial \tilde{u}_i}{\partial x_j} = -\frac{1}{\rho} \frac{\partial \tilde{p}}{\partial x_i} + \nu \frac{\partial^2 \tilde{u}_i}{\partial x_j^2} - \frac{\partial \tau_{ij}}{\partial x_j}, \quad (3)$$

where the overbar represents the low-pass filtering operation, $\tilde{u}_i$ is the filtered velocity, $\tilde{p}$ is the filtered pressure, ρ is the fluid (here, air) density, ν is the kinematic viscosity, and $\tau_{ij} = \tilde{u}_i \tilde{u}_j - \overline{u_i u_j}$ is the

subgrid-scale stress tensor representing the effects of the unresolved small scales on the resolved large scales.

To generate the training dataset for the neural network, large-eddy simulations (LES) of the airflow past the DrivAer geometry at 85 mph with no wheel rotation are performed using the low-dissipation CharLES solver. Details of the Voronoi diagram-based meshing paradigm used in the tool can be found in Brès *et al.*,²⁵ and the formulation of the low-Mach Helmholtz pressure solver is outlined in Ambo *et al.*²⁶ The code has been validated in the aerodynamics analysis of a number of high-Reynolds number flow studies.²⁷ The mesh uses 2×10^6 control volumes, with refinement near the no-slip surfaces of the car, as shown in Fig. 10. The resolution on the car surfaces is $\Delta/L = 0.0023$, where Δ is the mesh size locally, and L is the characteristic length of the car. The domain is

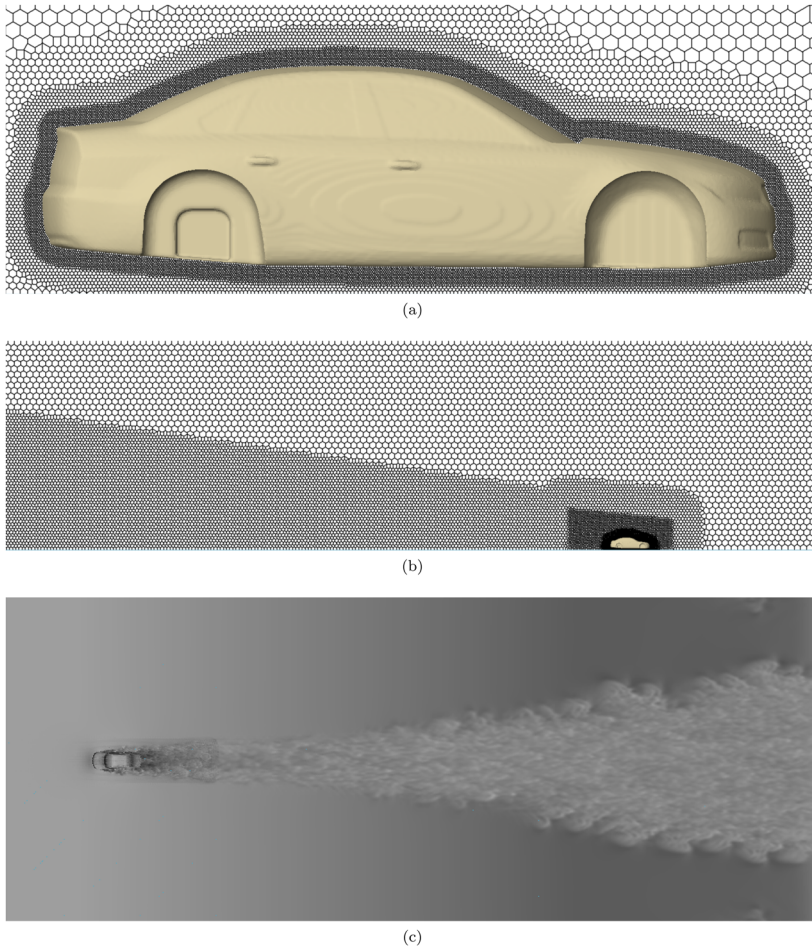


FIG. 10. View of the mesh used for the large-eddy simulations, with refinement zones shown: (a) near the car and (b) in the wake. (c) Instantaneous velocity magnitude contours showing the wake behind the car.

TABLE I. Simulation setup details for the DrivAerCH3 database.

Domain size	$15L \times 8L \times 6L$ (streamwise, spanwise, wall-normal)
Mesh size	2 MCVs
Wall model	Algebraic
Subgrid model	Vreman
Simulated time	2 s
Collected data	Averaged drag coefficient, pressure coefficients
Total number of simulations	1275

sized at $15L \times 8L \times 6L$, with the car placed at $5L$ from the inlet. Since the Helmholtz formulation permits low-frequency pressure oscillations, we use a numerical sponge at $2L$ from the domain boundaries to reduce the time for the reflecting waves to be damped out. The Vreman subgrid model is used to approximate the effects of the subgrid scales on the resolved scales, and walls are modeled using the algebraic wall model. The flow is initialized with a converged initial condition from the baseline DrivAer notchback geometry. The simulation is run for a total time of two seconds, and statistics are collected for the last one second. The simulation parameters are summarized in Table I. Figure 11 shows the instantaneous drag coefficient, computed as follows:

$$C_d = \frac{2F_d}{\rho v^2 A},$$

where F_d is the drag force, ρ is the fluid density, v is the freestream flow velocity, and A is the frontal area of the car projected in the direction of the flow. The initial spike is from the recalibration of the flow to the new boundary conditions (the new geometry), and the averaging begins well after equilibrium has been established.

The wall time for a single simulation to run to convergence is about 12 min on an NVIDIA H100 GPU.²⁸ A sample flow visualization depicting the velocity magnitude projected onto the floor of the domain is shown in Fig. 10, showing that the entire width of the wake is encapsulated in the domain. As mentioned above, care is taken to ensure that the mesh resolution at the wall is comparable

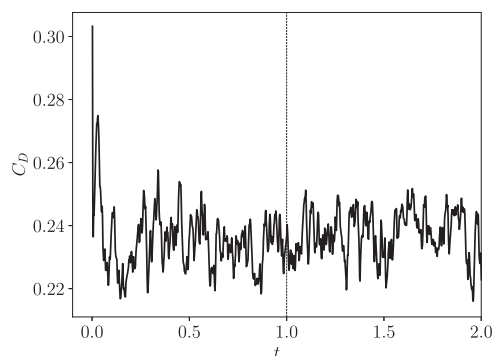


FIG. 11. Time history of drag coefficient for a single geometry. The dashed line indicates the start of the averaging window. The geometry and boundary conditions are to scale, so the time scaling is true to real-world time.

to the length scale introduced by the ray-tracing, which is $\sim L/1000$, where L is the length of the car, to ensure that discretization effects are consistent across different stages of the workflow. The simulation setup and parameters largely follow similar previous work on the same flow.²⁹ The effects of the ray-tracing are most evident in the reconstruction in regions of low but non-zero surface curvature, manifesting themselves as low-magnitude ripples, such as in the door panels in Fig. 10(a).

We make comparisons with the original set of experiments carried out by Heft *et al.*³⁰ The configuration that matches the one we use is the Fastback, with the Smooth underbody, without Mirrors, with Wheels (the $F_S_woM_wW$ experiment from Fig. 9 of Heft *et al.*) The reported C_D in that case is 0.227, which compares well with our value of 0.223. Our LES data match the experiments better than the Heft *et al.* RANS simulations, which give a value of 0.233. We note that the experiments in the paper were at a lower Reynolds number than ours ($7M$ vs $10M$); however, the C_D is observed to plateau around that Reynolds number, so we do not believe this has a significant impact. Our chosen Reynolds number was to correspond with typical highway speed limits. While the averaged drag coefficients are well-predicted with our meshes, the resolution is still insufficient to capture all energetic scales of interest. For instance, Nagaoka *et al.*³¹ used 66×10^6 control volumes in their study of the flow past an SUV. However, the purpose of this study is to use the DrivAerCH3 dataset to prototype the proposed shape interpolation strategy; thus, the resolution used is deemed sufficient, while not capturing localized quantities of interest with the accuracy requirements for practical design.

A. Network I/O

The choice of representation of the geometry is made evident when one considers that drag depends not on the internal design of the car, but only on the wetted surface, which scales as $\mathcal{O}(N^2)$ on a $\mathcal{O}(N^3)$ volume, where N is the characteristic number of points along one dimension of a discretized representation of the geometry. Thus, projections of the exterior surface are a natural representation choice for the geometry. For instance, Song *et al.*³² developed a surrogate model that used isometric views of automobile designs from the ShapeNet database.⁷ Projections can be naturally assembled into structured image data that can be processed by convolutional neural networks (CNNs).³³ CNNs are the most popular neural network architecture of choice for image processing, owing to the shared-weight model of the neurons, allowing for the same feature, say, an edge or sharp corner in an image, to be learned across the whole image by the same kernel. Fewer connections across a single layer of a network lead to a smaller network that is less prone to overfitting. CNNs are also advantageous from an ease-of-training standpoint relative to graph networks or point clouds,¹² the architecture of choice for unstructured representations. A graph representation of a geometry is also highly sensitive to small changes and is not an ideal way to represent a surface mesh, where the connections between two nodes do not have the same significance as in an actual graph network; that is, there can be several valid tessellations of the same geometry. Our inputs to the model are three planar projections of the geometry: the back, the side, and the top (shown in Fig. 12), stacked to make three channels. We note here that this choice is prompted by the fact that the geometries are identical in the other views, either

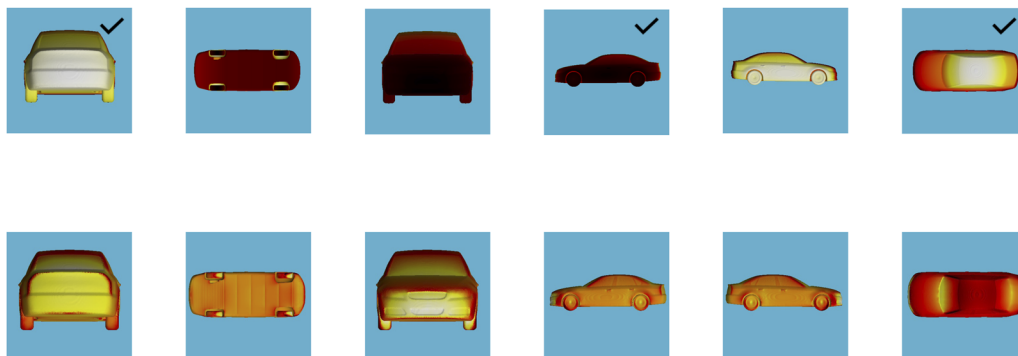


FIG. 12. Planar images of the normal distance from the camera (top row) and the normalized surface pressure coefficients (bottom rows) for a sample DrivAer geometry. The input images with the check marks (and the corresponding output views) are used for training.

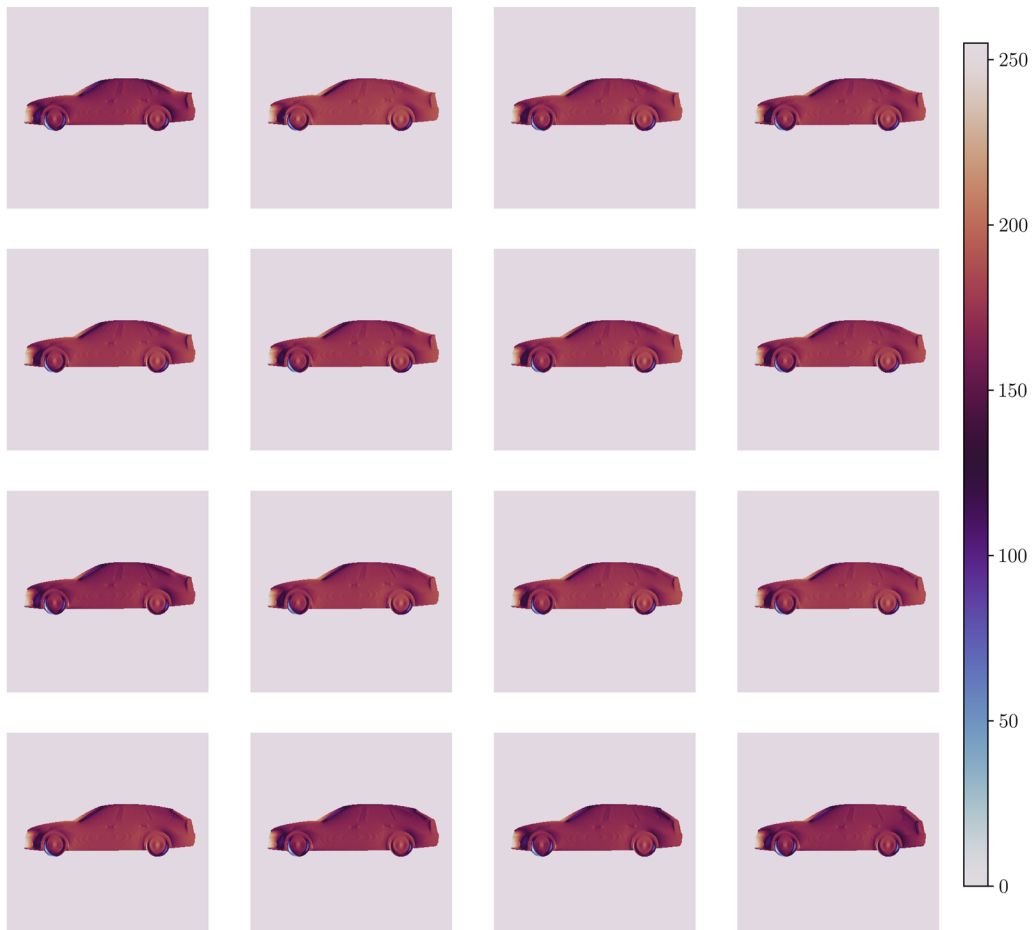


FIG. 13. Normalized surface pressure coefficients (c_p) shown on left side views of a selection of samples in the dataset, showing a transition from the notchback (top left) to the estateback (bottom right), with the fastback visited along the way.

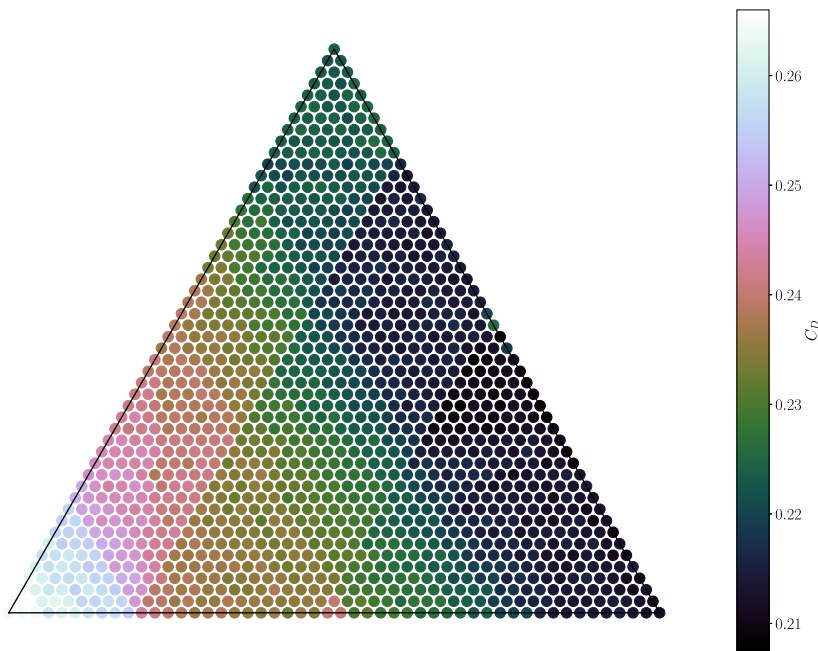


FIG. 14. Barycentric map showing drag coefficient of the DrivAerCH3 dataset with silhouettes of selected samples overlaid.

due to symmetry or the fact that the variation is restricted to the rear roofline. In general, we could use all six views (the current three, and the left side, bottom, and front views) to get maximum coverage. Each pixel records the shortest distance from the plane of the camera to the point on the vehicle. The image resolution is 384×384 , which is chosen to match the resolution of the mesh cells of the CFD and the resolution of the ray-tracing. This choice is made to resolve sufficiently fine changes to the surface. Concretely, taking the

different discretization choices (ray-tracing, mesh, and image resolution) into account, we expect to detect a change of about 12 mm to a surface in a vehicle that is 4 m long. The data in the images is binned into 255 bits.

While the drag coefficient is a useful average measure, design is driven by spatial information of pressure. We thus train networks to predict quantities of interest at two levels of abstraction: the scalar drag coefficient (C_D) and the full surface pressure coefficient (c_p)

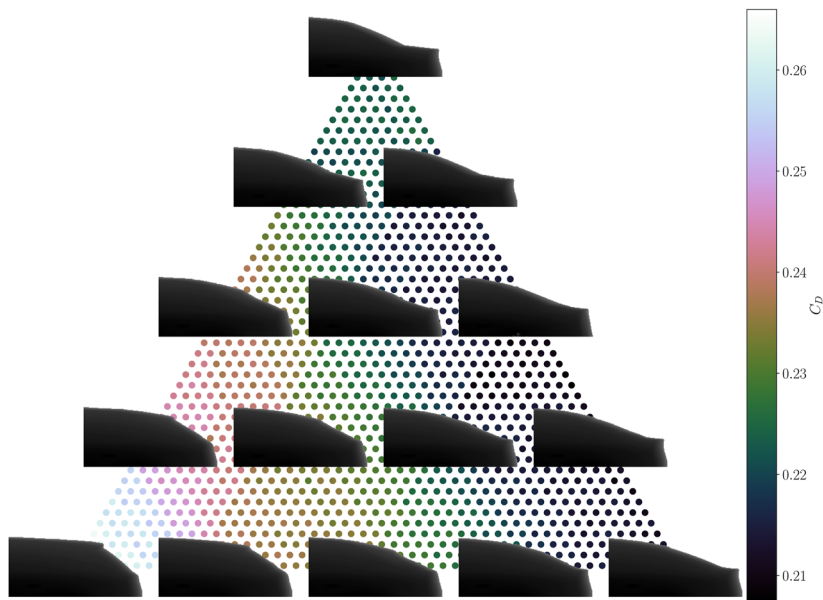


FIG. 15. Barycentric map showing drag coefficient of the DrivAerCH3 dataset.

TABLE II. Simulation and dataset parameters.

Mesh size	2M CVs	No. of basis cases	3
Wall model	Algebraic	No. of total samples	1275
Subgrid model	Vreman	Normalization	Min/max
Simulation time	2 s	Train/val split	1069/141

in the aforementioned three views. Individual neural networks are trained for each of these Quantities of Interest (QOI). Figure 13 shows a selection of samples from the dataset. One can see how the roofline is gradually morphed to produce designs intermediate to the basis cases. It is also evident how regions of low pressure are more apparent in the more estate-like designs, which is a result of the larger separation bubble that design tends to produce. With this method of generating training samples, the parameterization is not tied to discrete design features such as the door mirrors or handles,²⁹ but a comprehensive three-dimensional morphological operation, designed to provide a high density of data point clustering in feature space.

B. Dataset

The number of samples per dimension of the simplex is taken to be $n = 50$, which gives us $\frac{n(n+1)}{2} = 1275$ total geometries, for which we compute flow solutions as described above. The computational cost of generating the dataset is ~ 340 GPU-hours, ignoring meshing. Since the simplex in 3D is a triangle, it is convenient to plot each sample in a barycentric map, colored by its drag coefficient (Fig. 14). This plot provides an intuitive overview of the entire database, and the similarity of a given sample to a certain corner case is directly available as the Euclidean distance between their barycentric coordinates. Figure 15 shows the silhouettes of the rear upper sections of the cars overlaid on the triangle for selected samples. As one expects, the fastback (right vertex) has the lowest drag, followed by the notchback (top) and finally the estate (left). A summary of the

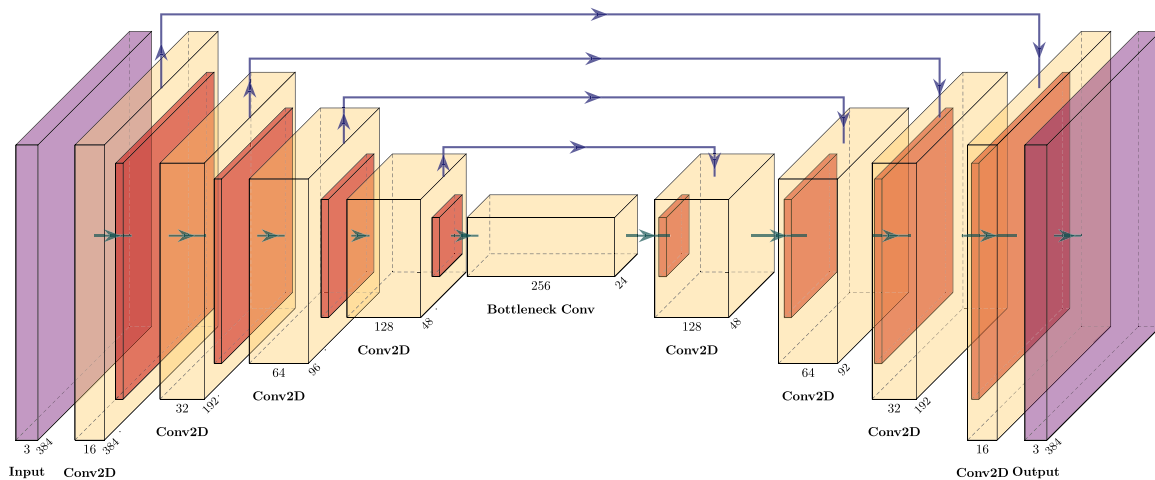
dataset—hereafter referred to as DrivAerCH3—is also provided in Table II.

C. Network architecture

The architecture of all the networks is convolutional. The two networks downsample the input of dimension $384 \times 384 \times 3$ down to a bottleneck of $24 \times 24 \times 256$ in the same way; however, the scalar C_D network has an output of size 1, both after going through a fully-connected layer of size 24. For the c_p case, we use a U-Net architecture³⁴ with symmetric skip connections, as shown in Fig. 16. The skip connections serve to greatly improve the recovery of localized information that may be lost in the several convolutional layers that the information passes through.³⁵ Maximum pooling layers are used after each convolutional layer to preserve and highlight important features. A summary of the model architecture is provided in Table III. Since the inputs to all three models are the same images, we keep the same architecture for the contracting part of the network, since the feature encoding is expected to be similar. One may envision this as a form of transfer learning, where the C_D model is tuned, and the architecture of the convolutional layers is retained for the c_p model. All layers, however, are trained from scratch in both models. The hyperparameters for the network—the learning rate, the number of filters per layer, and the optimizer type—are tuned by

TABLE III. Network architecture.

Architecture	CNN/U-Net
Activations	ReLU
Pooling	Max
Loss	MSE
Epochs	500
Optimizer	Adam
Learning rate	10^{-3}
No. of parameters	50K (1D model), 250K (2D), 1M (3D)

**FIG. 16.** Schematic of U-Net architecture for the tensor c_p model.

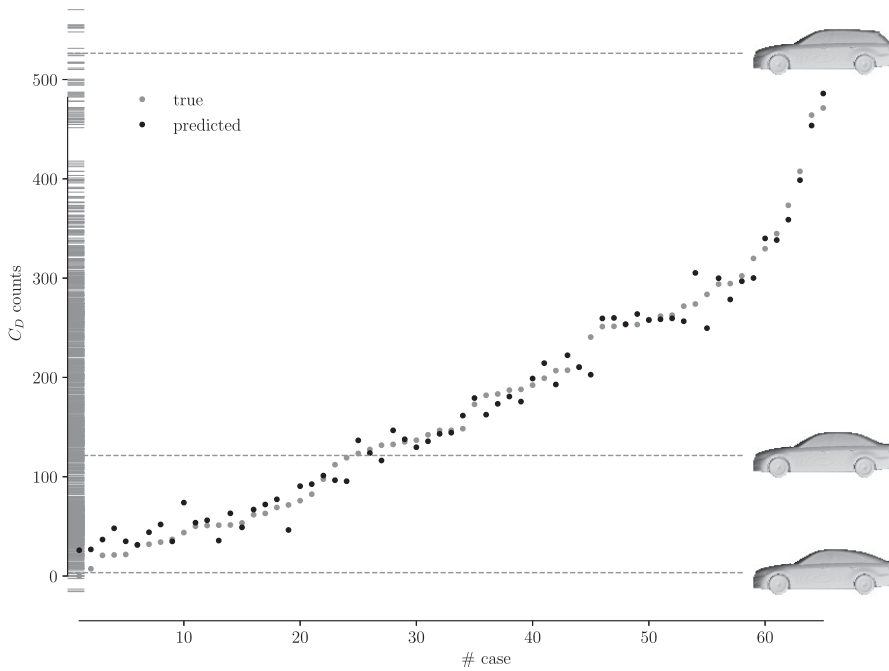


FIG. 17. C_D model performance on the test set, organized by increasing value of the ground truth. Horizontal dashed lines represent the reference drag coefficient values of the three basis geometries. The rug plot shows the distribution of the training set.

a grid search. The U-Net model, the biggest by parameter count, took 0.12 H100-hours to train to convergence. We perform aleatory and epistemic uncertainty quantification on the C_D networks by the method of ensembles, and this is reported in [Appendix B](#).

V. RESULTS AND DISCUSSION

A. Predicting C_D

[Figure 17](#) shows the performance of the model trained to predict the scalar drag coefficient on the test set consisting of 65 samples.

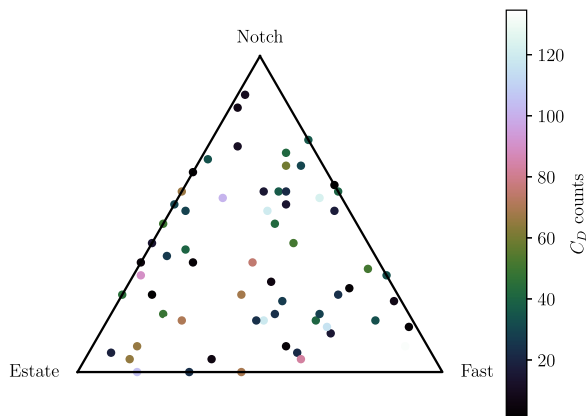


FIG. 18. C_D model averaged mean squared error (MSE) over test set for datasets of varying test set size.

The figure is rescaled by the lowest value of the ground truth and shows a range of about 500 drag counts of variation, defined as

$$\Delta C_D = C_D \times 10^4,$$

where C_D is the drag coefficient. The drag count simplifies comparisons across different configurations by scaling the drag coefficient to a more intuitive range. The measure is particularly useful when evaluating subtle changes in aerodynamic design. The network shows excellent predictive performance, with a 95th percentile accuracy of 24 counts. The controlled nature of the DrivAerCH3 dataset also allows us to plot the error on the barycentric map ([Fig. 18](#)), which gives us insight into the localization of the error. The quantity plotted is the relative error $\left| \frac{C_D^{\text{true}} - C_D^{\text{pred}}}{C_D^{\text{true}}} \right|$. While the current example does not show any error bias toward any of the three corners, it is evident that in cases where an initial equal sampling of data points in barycentric space produces an error response surface that contains regions of high error, a targeted resampling can be done, rather than a blind and expensive uniform increase in samples. To test the robustness of the results to the test set size, an additional set of experiments has been conducted: four additional datasets are constructed by considering test sets of increasing size: the first has 10% of the total data, the second 15%, the third 20%, and the fourth 25%. The validation splits for all these are kept fixed at 10% of the total data. New networks with the existing architecture are trained. [Figure 19](#) shows the average mean squared error (MSE) over the whole testing set for each of the four datasets, in addition to the original 5% split. We observe that there is not much variation in the results, with all the error values well within the same order of magnitude. This finding is also in agreement with what we have seen in the

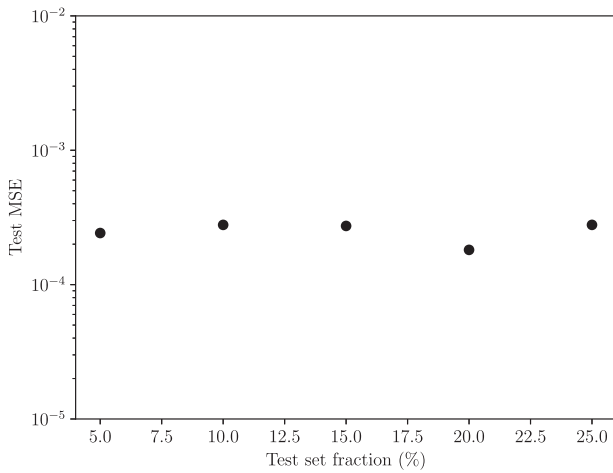


FIG. 19. Absolute error on the test set.

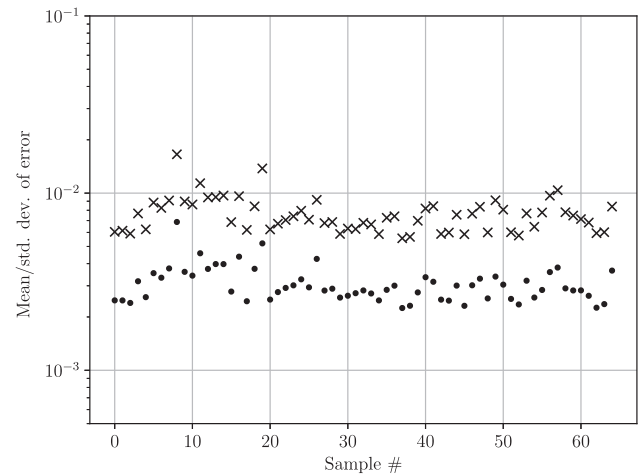


FIG. 21. MAE of the test set. Mean (dots) and standard deviation (crosses).

dataset ablation exercise from [Appendix A](#); for the C_D model, there is sufficient data for accurate predictions.

B. Predicting c_p

[Figure 20](#) shows the average error of spatial surface pressure coefficients over the whole testing set of 65 samples for the three different channels. The reconstructions are of good quality, with precise capture of localized features owing to the skip connections, which preserve memory across the central constriction. We see the presence of some crosstalk between the three images, showing that the three channels are not perfectly disentangled in the

training phase. However, the absolute magnitude of the error is low [$\mathcal{O}(10^{-3})$]. Areas of high concentration of the error include the trunk hatch, which is the region where the flow separates and the impact of the recirculation bubble on the surface pressure is seen, and the wheel arches. The mean and standard deviation of the error over the whole test set is shown in [Fig. 21](#); we see consistent performance across all samples, with a tracking of the two quantities.

We next show the performance of the networks for a single test set sample for both types of outputs. [Figure 22](#) shows the C_D model; [Fig. 23](#) shows that the c_p case is also reconstructed with good quality.

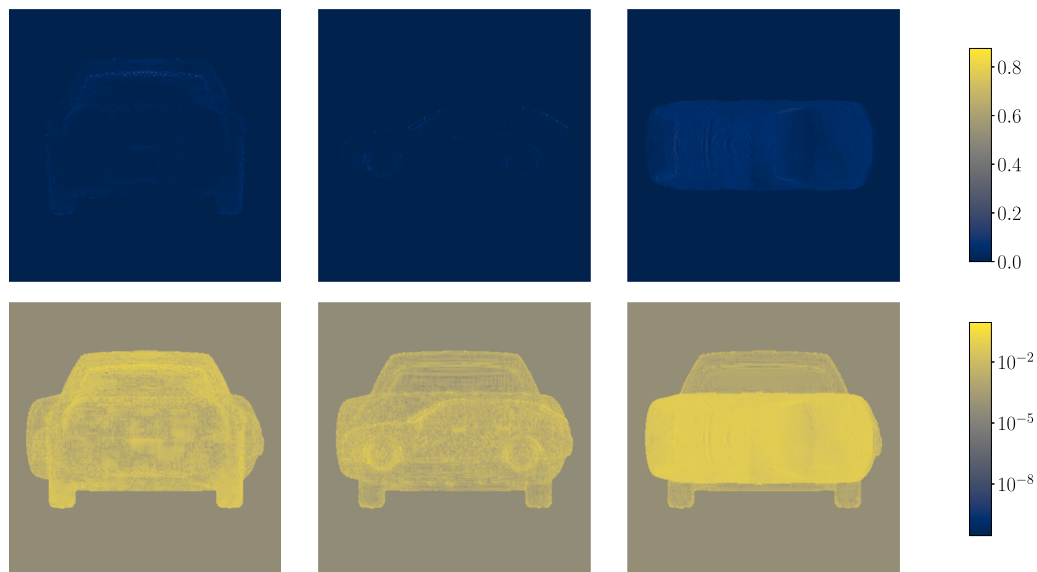


FIG. 20. Mean absolute error (MAE) of normalized surface pressure over the whole testing set. Left: channel 1; center: channel 2; right: channel 3. Top row: linear scale; bottom row: logarithmic scale.

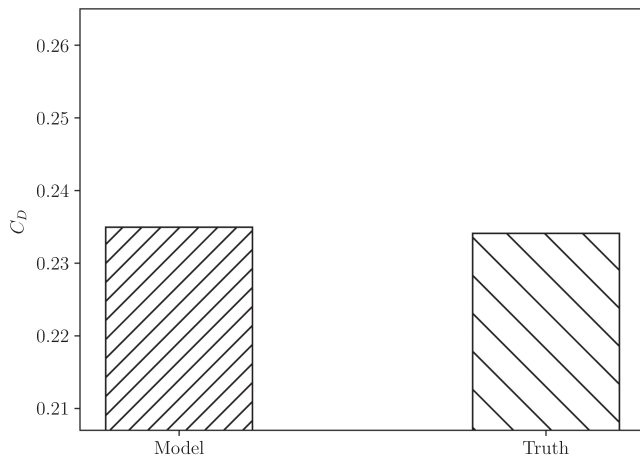


FIG. 22. Comparison of C_D error for one test set sample.

VI. NETWORK EXTRAPOLATION

One of the benefits of generating data within a simplex is that it allows for a direct low-dimensional visualization of different samples in relation to each other. It also allows for the study of network extrapolation in a controlled manner. Typically, extrapolation testing involves making inferences on samples from outside the training set distribution, with the idea being that good predictive performance on these unseen samples suggests that the network

is generalizing well. Using the ShapeCH procedure, we can make the process more systematic and introduce an *a priori* measure of extrapolation distance based on the inputs alone.

To do this, the following training-testing split of the DrivAerCH3 database is considered: the points inside the incircle of the barycentric triangle are used for training, and those outside are used for testing (see Fig. 24). Using this split, the testing set and training set are not drawn from the same distribution; in fact, the two sets are completely disjoint on the map. However, it is vital to note that the testing set is not completely disconnected from the training set: the two differ only by the ranges of the two free parameters used to generate the convex hull from the same basis cases. With this in mind, it becomes apparent that those samples in the testing set that lie closer to the circumference of the incircle are “closer” to the training set in the feature space and are expected to be less challenging to predict, with the amount of extrapolation increasing with increasing distance away from the incircle. Thus, it is possible to assign an “extrapolation distance” to each sample in the testing set, simply computed as the Euclidean distance from the circumference. This is shown in Fig. 25, expressed as a percentage ranging from 0% on the circle to 100% at the three vertices.

Next, a CNN to predict C_D (with similar specifications to the one above) is trained on the data inside the incircle. The expectation is that the network prediction error on the testing set will mirror the contours of Fig. 25. The error is shown in Fig. 26. It is seen that the lower left vertex, corresponding to the estateback basis case, has the spatial error distribution that is expected, with the error increasing radially outwards from the incircle and reaching a maximum

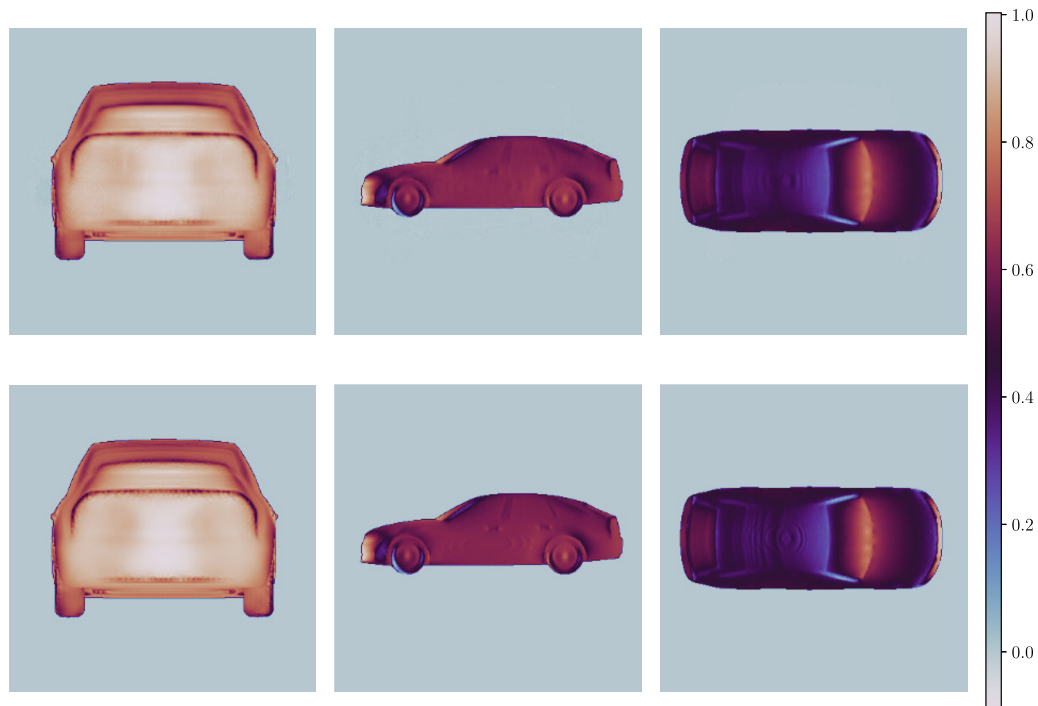


FIG. 23. Comparison of c_p error for one test set sample. Top row: model predictions; bottom row: ground truth.

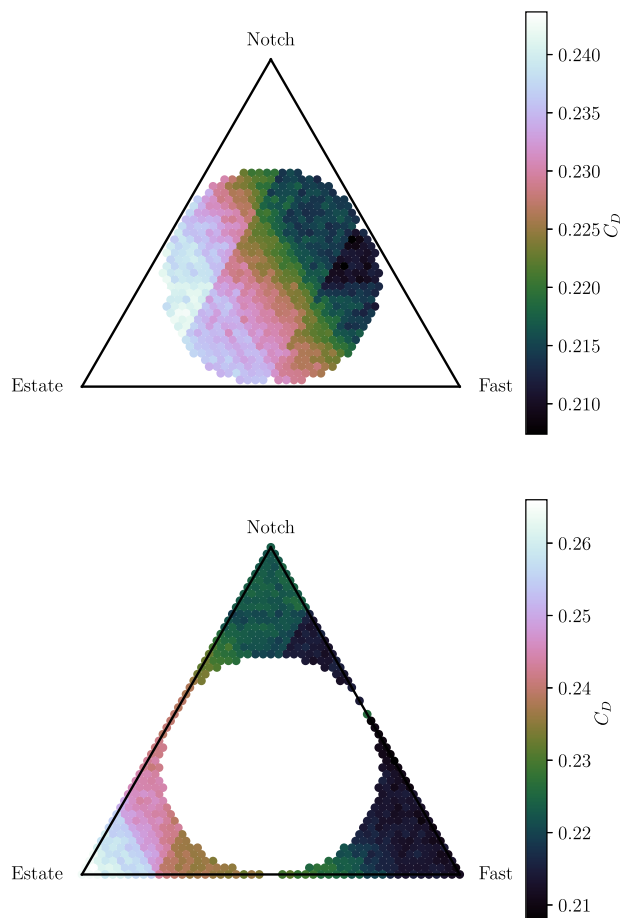


FIG. 24. Extrapolation study: training set (top) and test set (bottom), shaded by C_D .

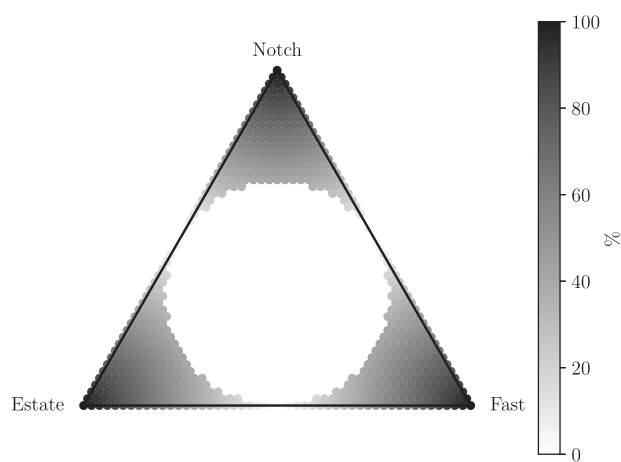


FIG. 25. Extrapolation distance (%).

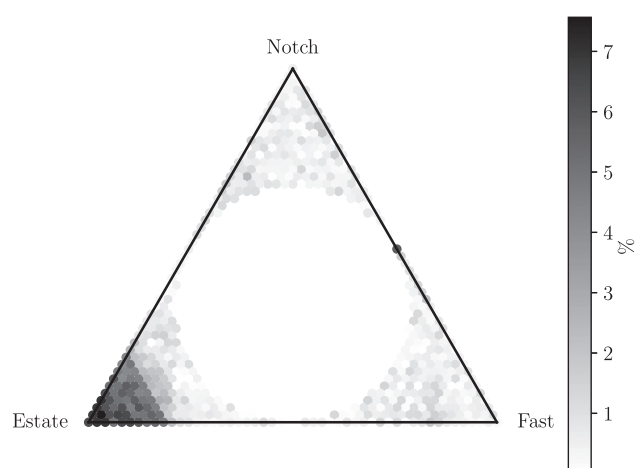


FIG. 26. Relative error (%) on test set.

at the vertex. However, this behavior is not seen in the vicinity of the other two vertices. The key to resolving this anomalous behavior is to recognize that the extrapolation distance does not account for the actual basis cases making up the vertices. In other words, Fig. 26 is independent of the database, given that there are three basis elements. It is clear that a full picture of extrapolation distance must include the effect of varying basis cases, so that if a certain vertex (or basis case) is much further away than the others in feature space, this is reflected by a corresponding increase in the extrapolation distance toward that vertex. This may be alternatively thought of as a rescaling of the sides of the triangle to reflect distances between the elements making up the vertices.

A. Effect of uniform barycentric sampling

To demonstrate the impact that a controlled sampling has on prediction quality, we examine a subset of 36 training samples and train two models: in the first, the points are sampled uniformly in barycentric space. In other words, the number of samples per dimension is 8, which gives a triangle number of 36. In the second model, the points are sampled randomly in barycentric space, to simulate an uncontrolled dataset. In this case, we test an ensemble of models with different random seeds and report a model with representative results. The models are trained to predict the scalar drag coefficient. The results from this experiment are shown in Fig. 27, on the same 65 test set samples from earlier experiments. We see that the effect of the controlled sampling is to produce a network with a high quality of predictions: the 95th percentile accuracy is 51 drag counts. The uniform sampling produces a network with insufficient coverage of the different portions of the simplex and has a much lower accuracy of 108 counts. This shows that even with a complex and non-linear response function, using equispaced training points enables better performance.

To understand the difference in performance, we investigate the loss landscape of the two models, by perturbing the network

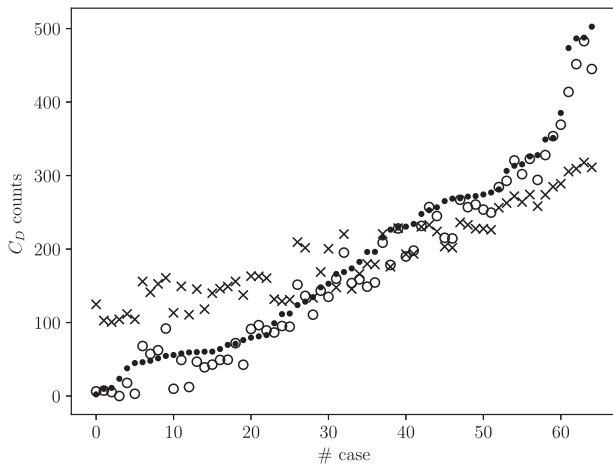


FIG. 27. C_D model performance with reduced dataset sizes. Dots: ground truth; circles: uniform barycentric sampling; crosses: random barycentric sampling.

weights. We apply perturbations to the weights of different magnitudes, and compute the corresponding change in the loss function. This allows us to visualize the stability and generalization potential of the networks. The maximum perturbation strength—0.01—is chosen to give an intermediate-scale picture of the loss landscape, between the overall shape, and the finest details. The loss landscapes for both the uniformly sampled and randomly sampled models are shown in Fig. 28. The analysis reveals that the uniform sampling results in a loss landscape with greater fluctuations, indicative of sharper minima and potentially less robust generalization. In contrast, the non-uniformly sampled model displays a smoother

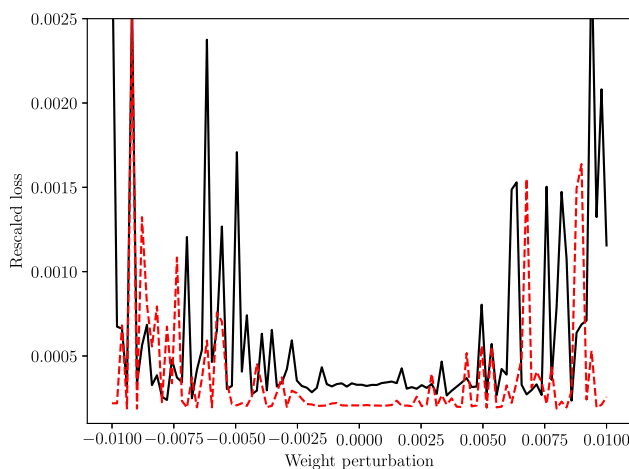


FIG. 28. Loss landscape comparison for C_D models; dashed red line: uniform barycentric sampling; solid black line: random barycentric sampling. The curves have their means subtracted out to better emphasize fluctuations.

landscape, suggesting a broader and more stable basin of minima, thereby indicating better generalization potential.

B. Comparison with other models: Gaussian process regression

The excellent results obtained by the CNN on the generated dataset prompt us to investigate the non-linearity of the problem: that is, is the prediction of aerodynamic drag on this dataset possible with a much simpler model? This question is best answered in an *a posteriori* setting, by comparing the performance of the CNN in the C_D case (the simplest) with another non-linear model. We avoid the obvious linear model (least-squares regression) because it is evident from Fig. 14 that the variation in drag coefficient over the barycentric map is non-linear. The most suitable reference for these investigations, therefore, is Gaussian Process Regression (GPR). GPR is a non-parametric, Bayesian inference model that offers a flexible approach to regression by defining a distribution over functions.³⁶ Since the purpose of this exercise is to investigate the complexity of the dataset, we directly feed the model the barycentric coordinates (x, y) that correspond to the geometries. This is another benefit to the database generation procedure we have outlined: each data point is identifiable uniquely by its position in the simplex.

The covariance function (or kernel) plays a critical role in the performance of GPR. We use a composite kernel $k(x_i, x_j) = \sigma_f^2 \exp(-\frac{1}{2\ell^2} \|x_i - x_j\|^2)$, where $\ell = 1.2$ is the length scale and $\sigma_f^2 = 1.0$ is the signal variance. The model was optimized by using the maximum likelihood framework for hyperparameter tuning, with ten restarts and a noise term $\alpha = 10^{-10}$. We observe from Fig. 29 that the tuned GPR model, while predicting the test set with fairly good accuracy (R^2 score 0.92), still struggles to match the performance of the CNNs. While the 95th percentile accuracy for the CNNs was 24 drag counts, it is 58 for the GPR. We reiterate that the variation of C_D across the three designs is primarily due to the separation

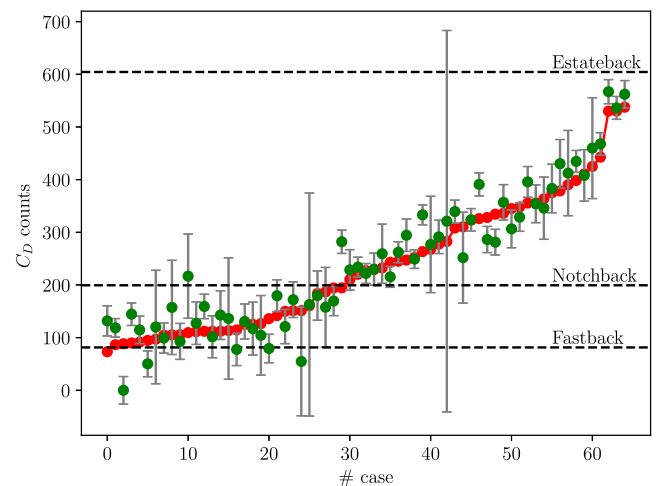


FIG. 29. Performance over test set. Red dots: ground truth; green dots: results from Gaussian Process Regression. The y-axis is zeroed using the minimum value of the GPR predictions. Error bars represent 95% confidence intervals.

from the roofline, which varies in a complex, three-dimensional way across the dataset. Thus, GPR requires an even higher sampling than the 50 points per dimension we have used in this case. Since the GPR models the prediction as a distribution, one can directly obtain uncertainty quantification bounds from the standard deviation of each output. This is also plotted in Fig. 29. A comparison with ensemble uncertainty quantification of the CNNs is shown in Appendix B, and one observes that the confidence intervals for GPR are much broader for the same dataset.

VII. CONCLUSIONS

A. Benefits of the proposed approach

In this work, we address the problem of the availability of high-quality geometries for training data-driven models for aerodynamics analysis. Focusing on automotive drag prediction, we develop a method by which a database of high-quality models can be systematically generated from a small number of starting designs. Our workflow involves the conversion of the STL representation of the geometry to an intermediate binary representation using an open-source ray-tracing tool (named ShapeCH) we have developed, followed by barycentric interpolation to generate new designs. The method has been tested using the three starting configurations of the DrivAer model to develop the DrivAerCH3 dataset, and we show that the performance on quantities of interest of various complexities—predicting scalar C_D and the spatial c_p —is excellent. Care has been taken to ensure that we are predicting variations in morphology on a sufficiently fine level, in keeping with practical considerations for automotive design. We attribute the performance of our models to the granularity of the dataset, which allows the networks to learn a smoother representation of the drag than would be possible with an unstructured dataset made up of an assortment of geometries. To show this, we conduct a *a posteriori* analysis of two networks trained on uniform and non-uniform barycentric sampling of the data, respectively, and show that the former has a smoother loss landscape, leading to more robust generalization and predictive accuracy. We believe this method could be the basis for a universal drag predictor, given enough training samples in a sufficiently high-dimensional space. Ongoing investigations are centered around designing a dataset that could potentially work with all cars of a certain type, for example, sedans, and the task is then to identify what makes a suitable corner geometry. It is of course evident that if two completely incompatible shapes are chosen for the corners, interpolation between them would result in shapes not likely to be used in practical design. However, the key is that the network learns about the surface pressure on these shapes regardless of whether or not the intermediate designs are practical. Other work in the field has used transfer learning to good effect; Song *et al.*,³² for instance, use a pre-trained ResNet for their investigations, with only a few trainable layers at the end. It is possible that as the databases grow in size, the training process could benefit from a network that has had broad exposure to a variety of inputs and can recognize fundamental image building blocks already. We are also interested in exploring other representations for the geometry. Images and CNNs have the benefit of ease of training but suffer from the issue of occlusion in non-convex geometries (which includes most automotive shapes), so alternative representation approaches such as Fourier Neural Networks (FNOs),³⁷ which have been used in conjunction with graph

representations of geometries,¹⁰ could be used. Our present workflow for generating geometries is easily compatible with any of these existing approaches. Another important piece of the problem is as follows: given a new, unseen geometry, what is the likelihood that the network prediction is within the expected accuracy based on the test set? In other words, how can we know that a given sample is out-of-distribution (OOD)? A solution for this might be to use convolutional autoencoders to use the same images we have used for the CNNs, to learn an encoding of the geometries in the dataset. Autoencoders used in this way have been successfully used to encode geometries in a parameter-free way.³⁸ This encoding—called a latent space representation—could then be used to determine whether or not a new geometry is OOD, and what the accuracy expected of the network can be. This ties in well with the planar projection representations of the present work, which work naturally with convolutional autoencoders, and will be an avenue for future investigations.

ACKNOWLEDGMENTS

Computations performed in this work were performed on the Kestrel Computing System at the National Renewable Energy Lab, Golden, CO. The authors gratefully acknowledge useful discussions with Dr. Frank Ham, Dr. Sanjeeb Bose, and Dr. Christopher Ivey.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Mark Benjamin: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Software (equal); Validation (equal); Visualization (equal). **Gianluca Iaccarino:** Conceptualization (equal); Formal analysis (equal); Funding acquisition (equal); Methodology (equal); Project administration (equal); Resources (equal); Supervision (equal); Writing – review & editing (equal).

DATA AVAILABILITY

Data and computer programs generated and used in this study are available at <https://zenodo.org/records/13334337>. The software developed in this work is released as an open-source project here: <https://github.com/benjamark/ShapeCH/>.

APPENDIX A: DATASET ABLATION

To get a sense of the amount of data required for this problem, we perform ablation studies for all three models. The training set is gradually reduced in size, keeping the validation and test sets fixed. The network architecture is left unchanged. The results in terms of average MSE over all the test set are shown in Fig. 30. The models converge at both levels of abstraction for ~200 samples, which corresponds to 20 samples per dimension. Below this, the c_p model degrades first. It is important to be sparse with the sampling in each dimension; with additional corner cases, the simplex becomes higher

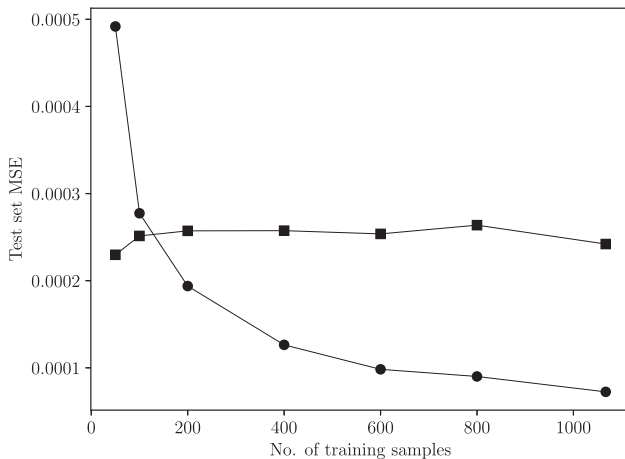


FIG. 30. MSE as a function of training set size.

dimensional and one is quickly confronted with the curse of dimensionality when ensuring sufficient coverage. It is also worth noting that the response surface of the drag coefficient can serve as a sensor for sampling; that is, regions of the simplex with a higher spatial gradient in drag coefficient values can be more densely sampled.

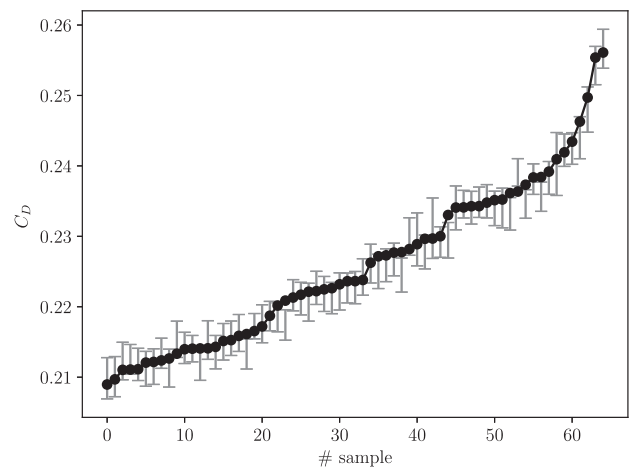
APPENDIX B: UNCERTAINTY QUANTIFICATION

To establish confidence in the predictions of the model, we investigate various sources of uncertainty, focusing on the C_D model. Various techniques have been developed to produce prediction confidence intervals for neural network-based models of fluid flow problems, such as the probabilistic neural networks proposed by Maulik *et al.*,³⁹ the Gaussian stochastic weight-averaging method of Morimoto *et al.*,⁴⁰ and methods that use Bayesian neural networks such as Sun and Wang.⁴¹ The method we follow is one of ensembles of models.⁴² We consider the following two types of input uncertainties:

- **Aleatory uncertainty:** Associated with the randomness in the learning process, this is not controllable. We consider two sources: the randomness of the initialization of weights in the model, which can have a significant impact on the converged optimum, and the selection of the batches used for training. The batch size is fixed at 64 after a hyperparameter tuning process, but there is still randomness associated with the elements in each batch, which affects the gradient computed at each step, and consequently the optimum.
- **Epistemic uncertainty:** This source of uncertainty is due to assumptions made in the model, and here we consider the choices made in the model architecture. We are interested in the impact that a perturbation to the size of the network can have, so we consider variations in the number of convolutional layers. Another important source of uncertainty is the impact that adding dropout layers can have, since this indicates how perturbations to the different connections in the network can impact the predictions. We note that the amount of dropout that is generally needed is much lower

TABLE IV. Details of UQ ensembles.

Parameter	Uncertainty type	Range
Weights initialization	Aleatory	n/a (seed)
Batch selection	Aleatory	n/a (seed)
Number of conv. layers	Epistemic	3–5
Dropout %	Epistemic	0.5–2

FIG. 31. C_D model 100% CI on test set. Black dots represent ground truth values.

in convolutional networks (as opposed to fully-connected ones), owing to the need for the identification of local features, and the fact that connections are sparser to begin with.

Table IV summarizes the variation in the ensemble of models. We follow the recommendations of Saetta *et al.*⁴³ in sizing the ensemble; they found that for a problem of similar scope, an ensemble size of 50 was sufficient to converge the uncertainty bounds. In the present work, we use 90 samples to build the ensemble.

Figure 31 shows the 100% confidence intervals of the models for all points in the testing set. We observe that there is a scatter of about 30 counts across the set (95th percentile), which is consistent with the accuracy of a single model with respect to the ground truth values.

REFERENCES

- ¹R. Palin, V. Johnston, S. Johnson, A. D'Hooge, B. Duncan, and J. I. Gargoloff, "The aerodynamic development of the Tesla Model S-Part 1: Overview," SAE Technical Report No. 2012-01-0177, 2012.
- ²J. Viquerat and E. Hachem, "A supervised neural network for drag prediction of arbitrary 2D shapes in laminar flows at low Reynolds number," *Comput. Fluids* **210**, 104645 (2020).
- ³R. Garcia-Fernandez, K. Portal-Porras, O. Irigaray, Z. Ansa, and U. Fernandez-Gamiz, "CNN-based flow field prediction for bus aerodynamics analysis," *Sci. Rep.* **13**(1), 21213 (2023).
- ⁴N. Umetani and B. Bickel, "Learning three-dimensional flow for interactive aerodynamic design," *ACM Trans. Graphics* **37**(4), 1 (2018).

- ⁵E. Gunpinar, U. C. Coskun, M. Ozsipahi, and S. Gunpinar, "A generative design and drag coefficient prediction system for sedan car side silhouettes based on computational fluid dynamics," *Comput.-Aided Des.* **111**, 65–79 (2019).
- ⁶B. Song, C. Yuan, F. Permenter, N. Arechiga, and F. Ahmed, "Surrogate modeling of car drag coefficient with depth and normal renderings," *arXiv:2306.06110* (2023).
- ⁷A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "ShapeNet: An information-rich 3D model repository," *arXiv:1512.03012* (2015).
- ⁸T. Rios, B. van Stein, P. Wollstadt, T. Bäck, B. Sendhoff, and S. Menzel, "Exploiting local geometric features in vehicle design optimization with 3D point cloud autoencoders," in *2021 IEEE Congress on Evolutionary Computation (CEC)* (IEEE, 2021), pp. 514–521.
- ⁹T. Rios, B. Sendhoff, S. Menzel, T. Bäck, and B. van Stein, "On the efficiency of a point cloud autoencoder as a geometric representation for shape optimization," in *2019 IEEE Symposium Series on Computational Intelligence (SSCI)* (IEEE, 2019), pp. 791–798.
- ¹⁰Z. Li, N. Borislavov Kovachki, C. Choy, B. Li, J. Kossaifi, S. Prakash Otta, M. Amin Nabian, M. Stadler, C. Hundt, K. Azizzadenesheli, and A. Anandkumar, "Geometry-informed neural operator for large-scale 3D PDEs," in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)* (NVIDIA, 2023).
- ¹¹E. Remelli, A. Lukoianov, S. Richter, B. Guillard, T. Bagautdinov, P. Baque, and P. Fua, "MeshSDF: Differentiable iso-surface extraction," in *Advances in Neural Information Processing Systems*, edited by H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Curran Associates, Inc., 2020), Vol. 33, pp. 22468–22478.
- ¹²M. Elrefaie, A. Dai, and F. Ahmed, "DrivAerNet: A parametric car dataset for data-driven aerodynamic design and graph-based drag prediction," *arXiv:2403.08055* [cs.LG] (2024).
- ¹³M. Elrefaie, F. Morar, A. Dai, and F. Ahmed, "DrivAerNet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks," *arXiv:2406.09624* (2024).
- ¹⁴M. Breuer, N. Jović, and K. Mazaev, "Comparison of DES, RANS and LES for the separated flow around a flat plate at high incidence," *Int. J. Numer. Methods Fluids* **41**(4), 357–388 (2003).
- ¹⁵S. T. Bose and G. I. Park, "Wall-modeled large-eddy simulation for complex turbulent flows," *Annu. Rev. Fluid Mech.* **50**, 535–561 (2018).
- ¹⁶J. Svorcan and C. Ivey, "WMLES of a small-scale hovering propeller," in *International Symposium on Unmanned Systems and the Defense Industry* (Springer, 2022), pp. 259–264.
- ¹⁷K. Goc, S. Bose, and P. Moin, "Wall-modeled large eddy simulation of an aircraft in landing configuration," AIAA Paper 2020-3002, 2020.
- ¹⁸A. S. Ghate, J. A. Housman, G.-D. Stich, G. Kenway, and C. C. Kiris, "Scale resolving simulations of the NASA juncture flow model using the lava solver," AIAA Paper 2020-2735, 2020.
- ¹⁹S. Osher and R. Fedkiw, *Level Set Methods and Dynamic Implicit Surfaces* (Springer, 2003).
- ²⁰T. Möller and B. Trumbore, "Fast, minimum storage ray/triangle intersection," in *ACM SIGGRAPH 2005 Courses* (Association for Computing Machinery, 2005).
- ²¹W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques* (Association for Computing Machinery, 1987), pp. 163–169.
- ²²O. Sorkine, "Laplacian mesh processing," in *Eurographics 2005: State of the Art Reports*, edited by Y. Chrysanthou and M. Magnor (The Eurographics Association, 2005).
- ²³A. Heft, T. Indinger, and N. Adams, "Introduction of a new realistic generic car model for aerodynamic investigations," SAE Technical Report No. 2012-01-0168, 2012.
- ²⁴M. Aultman, Z. Wang, R. Auza-Gutierrez, and L. Duan, "Evaluation of CFD methodologies for prediction of flows around simplified and complex automotive models," *Comput. Fluids* **236**, 105297 (2022).
- ²⁵G. Brès, S. Bose, M. Emory, F. Ham, O. Schmidt, G. Rigas, and T. Colonius, "Large-eddy simulations of co-annular turbulent jet using a Voronoi-based mesh generation framework," AIAA Paper 2018-3302, 2018.
- ²⁶K. Ambo, H. Nagaoka, D. A. Philips, C. Ivey, G. A. Brès, and S. T. Bose, "Aerodynamic force prediction of the laminar to turbulent flow transition around the front bumper of the vehicle using dynamic-slip wall model LES," AIAA Paper 2018-3302, 2020.
- ²⁷K. A. Goc, O. Lehmkuhl, G. I. Park, S. T. Bose, and P. Moin, "Large eddy simulation of aircraft at affordable cost: A milestone in computational fluid dynamics," *Flow* **1**, E14 (2021).
- ²⁸J. Choquette, "NVIDIA hopper H100 GPU: Scaling performance," *IEEE Micro* **43**(3), 9–17 (2023).
- ²⁹M. Benjamin, E. Saetta, C. Ivey, S. Bose, F. Ham, and G. Iaccarino, "Parametric representation and surrogate modeling of drag for a realistic automotive geometry," in *Center for Turbulence Research Annual Research Briefs* (Center for Turbulence Research, 2023).
- ³⁰A. I. Heft, T. Indinger, and N. A. Adams, "Experimental and numerical investigation of the DrivAer model," in *Fluids Engineering Division Summer Meeting* (American Society of Mechanical Engineers, 2012), Vol. 44755, pp. 41–51.
- ³¹H. Nagaoka, B. Yenerdag, K. Ambo, D. Philips, C. Ivey, G. Brès, and S. Bose, "Prediction of aerodynamic drag in SUVs with different specifications by using large-eddy simulations," SAE Technical Report No. 2024-01-2525, 2024.
- ³²B. Song, C. Yuan, F. Permenter, N. Arechiga, and F. Ahmed, "Surrogate modeling of car drag coefficient with depth and normal renderings," *arXiv:2306.06110* (2023).
- ³³K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *arXiv:1511.08458* (2015).
- ³⁴O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *arXiv:1505.04597* (2015).
- ³⁵G. Xu, X. Wang, X. Wu, X. Leng, and Y. Xu, "Development of skip connection in deep neural networks for computer vision and medical image analysis: A survey," *arXiv:2405.01725* [eess.IV] (2024).
- ³⁶C. Williams and C. Rasmussen, "Gaussian processes for regression," in *Advances in Neural Information Processing Systems*, edited by D. Touretzky, M. C. Mozer, and M. Hasselmo (MIT Press, 1995), Vol. 8.
- ³⁷Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," *arXiv:2010.08895* (2020).
- ³⁸E. Saetta, R. Tognaccini, and G. Iaccarino, "Machine learning to predict aerodynamic stall," *Int. J. Comput. Fluid Dyn.* **36**(7), 641–654 (2022).
- ³⁹R. Maulik, K. Fukami, N. Ramachandra, K. Fukagata, and K. Taira, "Probabilistic neural networks for fluid flow surrogate modeling and data recovery," *Phys. Rev. Fluids* **5**, 104401 (2020).
- ⁴⁰M. Morimoto, K. Fukami, R. Maulik, R. Vinuesa, and K. Fukagata, "Assessments of epistemic uncertainty using Gaussian stochastic weight averaging for fluid-flow regression," *Physica D* **440**, 133454 (2022).
- ⁴¹L. Sun and J.-X. Wang, "Physics-constrained Bayesian neural network for fluid flow reconstruction with sparse and noisy data," *Theor. Appl. Mech. Lett.* **10**(3), 161–169 (2020).
- ⁴²W. He, Z. Jiang, T. Xiao, Z. Xu, and Y. Li, "A survey on uncertainty quantification methods for deep learning," *arXiv:2302.13425* [cs.LG] (2024).
- ⁴³E. Saetta, R. Tognaccini, and G. Iaccarino, "Uncertainty quantification in autoencoders predictions: Applications in aerodynamics," *J. Comput. Phys.* **506**, 112951 (2024).